Hydro Pulse

Roteiro completo de estudo e apresentação

Apresentação prevista para 26 de outubro de 2026. Aplicativo 3.0.4, código Android 12; API 3.0.1. Material para Matheus Felipe dos Santos, Roger Alexey Camargo Lamim e Vinícius da Silva Ribeiro.

O Hydro Pulse reúne histórico oficial de chuva em Santos e relatos da comunidade. Seu principal resultado acadêmico é demonstrar como o celular e o servidor continuam preservando informações durante falhas de comunicação e como voltam a compartilhar dados depois da reconexão.

Este roteiro começa pela fala e pela demonstração. Depois explica as telas, os arquivos, os contratos da API e o banco. As dez perguntas finais servem para ensaiar a defesa. Não é necessário decorar cada linha: procure acompanhar o caminho de um relato, do formulário ao banco e de volta ao aplicativo.

Se você tem pouco tempo: estude as seções 1 e 2, entenda a sequência da seção 7, memorize o exemplo do cursor na seção 8 e responda às dez perguntas sem ler. Os caminhos de código usados neste material partem da pasta codigo do ZIP enxuto. Os materiais prontos ficam em materiais e os documentos de apoio em documentos. As suítes de testes foram retiradas desta cópia; seus resultados registrados foram preservados.

Conteúdo

1 O projeto em uma explicação curta 2
2 Roteiro de fala e demonstração 2
3 A proposta original e o que foi implementado 4
4 Arquitetura e conceitos de sistemas distribuídos 5
5 Tecnologias e motivo das escolhas 6
6 Telas e navegação 6
7 O caminho completo de um relato 7
8 Sincronização e recuperação por cursor 8
9 API e contratos HTTP 9
10 Banco de dados central 11
11 Conta segurança e privacidade no código 12
12 Mapa localização e imagens 13
13 Dados oficiais de chuva 14
14 Confirmações contestações e encerramento 15
15 Como apresentar o código sem se perder 15
16 Validação registrada 18
17 Como preparar a execução amanhã 18
18 Operação manutenção e limites da entrega 19
19 Revisão rápida e glossário 20
20 Dez perguntas que o professor pode fazer 22

1 O projeto em uma explicação curta

Fala sugerida: “O Hydro Pulse é uma aplicação Android para consultar informações oficiais de chuva em Santos e compartilhar relatos de alagamento. O celular guarda os dados já recebidos e os relatos ainda não enviados. Se a rede falhar, uma ocorrência fica salva no aparelho; quando a comunicação volta, ela é enviada ao servidor. O servidor reconhece os reenvios pelo mesmo identificador e evita criar outro registro para a mesma operação.”

Complete com a finalidade da disciplina: “Nós usamos esse problema para demonstrar persistência local, comunicação cliente-servidor, falhas parciais, idempotência e recuperação de atualizações. A chuva oficial e os relatos colaborativos têm origens diferentes e aparecem identificados.”

Três afirmações que precisam estar claras: uma medida de chuva não comprova uma rua alagada; um relato salvo localmente ainda pode não estar no servidor; uma confirmação da comunidade não equivale à validação de uma autoridade.

O produto entregue tem navegação e funcionalidades para um usuário comum. A execução combinada para a aula usa a API no notebook e um túnel temporário HTTPS. Hospedagem permanente e publicação em loja são etapas separadas da aplicação implementada.

2 Roteiro de fala e demonstração

Organização sugerida para 12 a 15 minutos

Tempo	Quem pode apresentar	O que mostrar e explicar
0 a 1 min	Matheus	Problema, recorte em Santos e objetivo do projeto.
1 a 3 min	Roger	Celular, API e SQLite; por que a arquitetura é distribuída.
3 a 4 min	Matheus	Fonte oficial, período dos dados e diferença entre medição e consulta.
4 a 8 min	Vinícius	Mapa, formulário, salvar offline, reabrir e sincronizar.
8 a 11 min	Roger	Caminho do código, reenvio sem duplicação e cursor.
11 a 13 min	Vinícius e Matheus	Conta, encerramento, confiança nos relatos e testes.
13 a 15 min	Todos	Resultados, limites e fechamento.

Essa divisão acompanha as áreas previstas na proposta. É uma sugestão de fala e estudo, sem atribuir a cada integrante uma autoria de implementação que não esteja documentada. Cada pessoa deve compreender o fluxo completo para responder às perguntas.

Abertura e passagem para a arquitetura

Fala: “Nosso recorte é Santos. Reunimos dados oficiais de precipitação e observações feitas por pessoas. O objetivo é permitir a consulta e a colaboração mesmo quando a comunicação é intermitente. Não estamos prevendo alagamentos nem substituindo a Defesa Civil.”

Mostrar: Início e Medições reais de chuva. **Explicar:** o dado institucional informa chuva por data; um relato tem autor, categoria, horário observado e posição. Passe a palavra: “Para sustentar isso, o sistema foi dividido entre aplicativo e servidor.”

Arquitetura em linguagem simples

> **Fala:** “O aplicativo é a parte com a qual o usuário interage. A API recebe as requisições, valida as informações e decide o que pode ser gravado. O SQLite do servidor mantém o estado compartilhado. No celular, o AsyncStorage conserva o cache e as pendências, e o SecureStore protege o token da sessão no Android.”

Mostrar: o desenho da seção 4 e os arquivos `mobile/App.tsx`, `mobile/src/store/appState.ts` e `server/src/app.ts`. Não abra dezenas de arquivos ao mesmo tempo: mantenha esses três e `server/src/db/reports.ts` preparados.

Demonstração principal no celular

1. Entre previamente na conta de ensaio. Mostre Início e a origem das medições. Na cópia distribuída há 22 datas de 01 a 22/10/2026, com soma de 169,7 mm. Leia a data que o app efetivamente apresentar se houver atualização.

2. Abra Mapa, amplie uma rua e mostre a legenda. Em Relatar, escolha um ponto no mapa ou use sua localização com permissão. O endereço escrito é descritivo; as coordenadas vêm do ponto escolhido.

3. Preencha categoria e descrição “DEMONSTRAÇÃO - ensaio acadêmico”. Se usar foto, escolha uma imagem de ensaio. Deixe o formulário pronto antes de desligar a rede.

4. Ative modo avião e desligue também o Wi-Fi. Envie o relato. Mostre que ele fica aguardando envio, em vez de afirmar que já foi compartilhado.

5. Feche o app pela lista de recentes e abra novamente. Em Atividade, confira o relato, o ponto e a foto. **Fala:** “A tela foi fechada, mas a operação permaneceu no armazenamento do aparelho.”

6. Reative a internet e toque em Sincronizar agora. Mostre a mudança para enviado/sincronizado e o relato no mapa. **Fala:** “Agora o servidor recebeu a ocorrência. O mesmo identificador acompanha a operação desde antes do primeiro envio.”

7. Se houver tempo, abra o relato e marque a ocorrência como encerrada, com uma nota de demonstração. No mapa, habilite a exibição de encerradas; o marcador fica cinza. Explique que encerramento e moderação são estados diferentes.

Demonstração do reenvio no computador

Na pasta codigo, com as dependências preparadas, execute:

```
npm run demo:idempotency
```

> **Fala:** “Esse comando de demonstração cria um banco temporário e provoca uma falha depois de o servidor gravar. A primeira resposta se perde. A segunda tentativa mantém a mesma identidade da operação. O resultado esperado é duas tentativas e uma única linha no banco.”

Mostre `attempts: 2` e `databaseRows: 1`. O teste chama a rota canônica de relatos do mesmo servidor; ele demonstra a regra central de gravação usada também pelo contrato do mobile. Não é preciso derrubar a API da apresentação nem adicionar dados de ensaio ao banco utilizado pelo celular.

Fechamento sugerido

Fala: “Entregamos aplicativo, API, persistência, dados oficiais identificados e testes de falhas e reconexão. O núcleo demonstra que o cliente consegue guardar uma operação, repetir seu envio sem duplicação lógica e recuperar dados do servidor. Os limites atuais incluem o endereço temporário da aula, o mapa dependente de internet ou cache e a ausência de notificações push.”

Se o professor reduzir o tempo, mantenha objetivo, arquitetura, demonstração offline e explicação da idempotência. Esses quatro pontos mostram o vínculo mais direto com Sistemas Distribuídos.

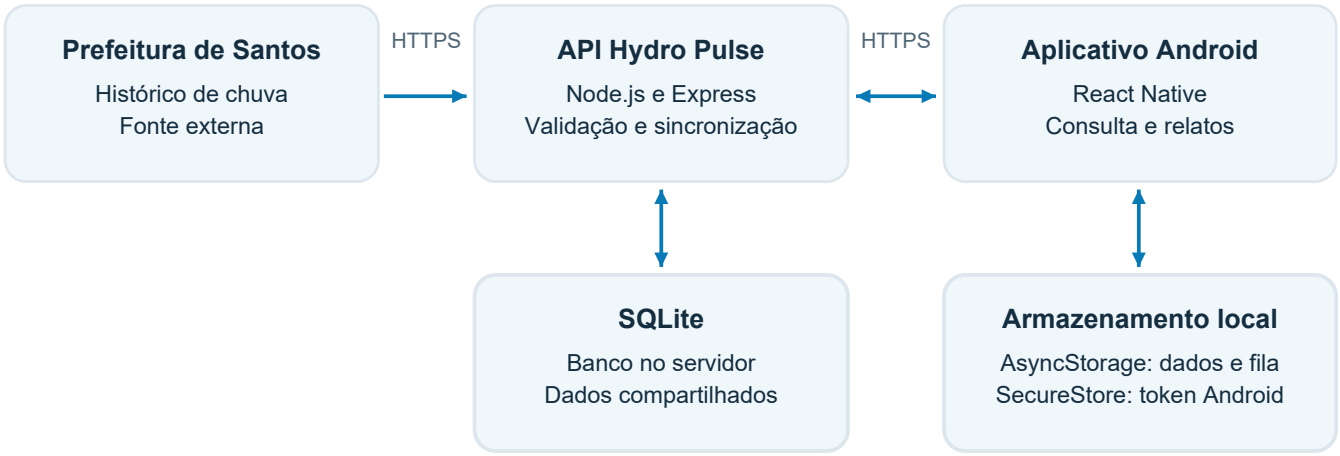
3 A proposta original e o que foi implementado

O PDF é uma proposta inicial, não uma declaração de resultados já obtidos. Ele prioriza consulta de fonte institucional, relatos, persistência, reenvio e recuperação. Permite ajustes de tecnologia preservando os objetivos acadêmicos e aceita importação controlada de registros oficiais quando a integração disponível for limitada.

Ponto da proposta	Implementação atual e explicação para a banca
Consulta de dados reais com procedência	Histórico da Prefeitura de Santos, importação validada e consulta automática da página oficial. Fonte, unidade, abrangência e datas são preservadas.
Mapa e dados com granularidade correta	Ruas OpenStreetMap e pontos reais dos relatos. O histórico municipal não fornece coordenadas de estações; a API retorna a lista de estações vazia.
Registro durante desconexão	Relato e imagem preparada ficam no estado persistido do aparelho antes do envio. Reabrir recupera a fila.
Reenvio sem duplicação	`clientId` estável no mobile, cabeçalho `Idempotency-Key`, unicidade no banco e comparação do conteúdo.
Recuperação após ficar offline	Consulta incremental por cursor e atualização separada dos dados da comunidade.
SQLite previsto no celular	A implementação mantém o AsyncStorage do mobile fornecido, com JSON e fila de gravação. SQLite está no servidor. O objetivo de persistência local continua atendido, sem afirmar que o celular executa SQL.
Fontes Cemaden ou Prefeitura	A integração efetiva desta entrega usa Prefeitura de Santos. A referência ao Cemaden na proposta não significa uma integração ativa.
Push como complemento opcional	Foram implementados avisos internos recebidos ao sincronizar. Expo Push e FCM não são usados no fluxo entregue.
Validação com dois clientes	Há testes com clientes independentes e ensaio físico em um Android com contraparte HTTP. Não se declara teste final em dois celulares Android.
Funcionalidades secundárias	Conta, recuperação por chave, confirmações, contestações, encerramento e moderação ampliam o núcleo original.

Na proposta, use as páginas físicas 4 a 11 do PDF: objetivo e escopo nas páginas 4 a 6; tecnologias nas páginas 6 e 7; dados e offline na página 8; validação nas páginas 9 e 10; divisão inicial e resultado nas páginas 10 e 11. A numeração impressa começa antes: a página física 4 está marcada como página 3.

4 Arquitetura e conceitos de sistemas distribuídos



OpenStreetMap fornece as ruas ao app por uma conexão separada.

O telefone, o servidor e a fonte institucional executam independentemente e se comunicam pela rede. O OpenStreetMap é outra dependência externa, usada para as imagens das ruas. O SQLite central é acessado pelo servidor; o celular fala com a API, sem abrir o arquivo do banco central.

Por que há distribuição mesmo com um só servidor: os clientes estão em outros dispositivos, possuem estado local e podem ficar inacessíveis enquanto o servidor continua recebendo dados. A fonte oficial também pode falhar enquanto o app continua exibindo o último histórico válido. Não é necessário ter microsserviços ou um cluster para demonstrar distribuição.

Falha parcial: um componente ou caminho de comunicação falha e outros continuam funcionando. Exemplos concretos: há internet no celular, mas a API está indisponível; o servidor grava e a resposta não chega; a Prefeitura falha, mas a base anterior permanece consultável.

Consistência eventual: durante a desconexão, o aparelho e o servidor podem ter estados diferentes. O app mostra a pendência e tenta compartilhar depois. A convergência depende da comunicação voltar, da sessão ser válida e das operações serem aceitas. Isso não significa que todos os aparelhos veem tudo imediatamente.

Tolerância à desconexão: a gravação local continua disponível para uma sessão já aberta. Primeiro login, cadastro, recuperação de conta e novas consultas remotas precisam de rede. Não há promessa de sincronização contínua com o app totalmente fechado.

Transação local do banco: agrupa gravações do servidor para concluir a operação de forma consistente. Não é uma transação distribuída entre celular e servidor. A fila local e a idempotência tratam a separação entre esses dois lados.

Limite arquitetural: há um servidor e um arquivo SQLite central. Não foram implementados consenso distribuído, replicação entre servidores ou alta disponibilidade. Se esse servidor parar, o compartilhamento remoto espera; o armazenamento local continua sendo útil.

5 Tecnologias e motivo das escolhas

Tecnologia	Papel no código	Como explicar
React Native e Expo	Telas Android, recursos do aparelho e geração do APK	Permitem organizar a interface em componentes e usar localização, imagens e armazenamento. O APK compilado contém o JavaScript e não precisa de Expo aberto para uso normal.
TypeScript	Tipos no app e na API	Ajuda a detectar incompatibilidades durante o desenvolvimento. Dados recebidos da rede ainda precisam de validação em execução.
React Navigation	Pilha de telas e cinco abas	Separa acesso sem sessão, uso autenticado e rotas administrativas.
Zustand	Estado central no celular	Reúne usuário, dados, pendências, cursor e ações, acessados pelas telas.
AsyncStorage e SecureStore	Persistência Android	O primeiro guarda JSON; o segundo guarda o token separado do estado comum. AsyncStorage não é banco SQL nem criptografia geral dos relatos.
Node.js 24 e Express 5	Processo do servidor e rotas HTTP	Recebem requisições JSON, validam permissões e coordenam gravações e consultas.
Zod	Validação dos contratos	Confere formato, tamanho, categorias, coordenadas e outros valores recebidos.
SQLite e better-sqlite3	Banco central	Simplificam a instalação e oferecem transações, restrições de unicidade e consultas SQL. Não substituem uma arquitetura dimensionada para grande volume.
Leaflet e OpenStreetMap	Motor do mapa e imagens das ruas	Leaflet desenha e movimenta o mapa; OpenStreetMap fornece os tiles e a atribuição de origem.
WebView e iframe	Integração do mapa com cada plataforma	O Android usa WebView; a versão web usa iframe para o mesmo documento do mapa.
Cloudflare Tunnel	Caminho HTTPS até a API local	Torna a API do notebook acessível ao celular durante o ensaio. O túnel não hospeda nem armazena o banco.

As versões da entrega são diferentes porque o patch 3.0.4 reorganizou a interface. A API permanece na versão 3.0.1, compatível com o mesmo contrato.

6 Telas e navegação

`mobile/App.tsx` monta a aplicação. Primeiro aguarda a recuperação dos dados locais, chamada de hidratação. Se houver erro de leitura, mostra uma mensagem e permite tentar de novo. Se existir uma chave de recuperação recém-gerada para exibir, apresenta essa etapa antes da navegação normal.

Sem sessão há acesso, cadastro e recuperação em AuthScreen, consulta pública de chuva e Configurar conexão. A URL só fica disponível nessa configuração após informar 180504. Com sessão aparecem as cinco abas; moderação e a configuração administrativa do Perfil exigem papel moderator.

Tela ou conjunto	Responsabilidade
Início e Medições reais de chuva	Resumo, data do histórico, atalhos, tabela de índices, procedência e última consulta.
Mapa e Escolher local	Ruas, filtros, pontos, localização do aparelho e seleção de uma coordenada para o relato.
Relatar e Sucesso	Categoria, descrição, endereço, horário observado, ponto e foto opcional; resultado da gravação/envio.
Detalhe e Edição de relato	Detalhes, confirmação, contestação e situação da ocorrência. A edição do conteúdo é limitada ao relato ainda pendente e não tentado.
Atividade	Estado da conexão, sincronização manual, fila e histórico recebido.
Meu perfil e Editar perfil	Identidade primeiro, dados pessoais e menus agrupados. O nome de exibição pode mudar sem trocar o nome de acesso.
Conta e segurança e Chave de recuperação	Troca de senha, geração de chave, exclusão e apresentação temporária da chave.
Notificações	Avisos recebidos na sincronização e navegação ao relato relacionado.
Administração	Análise de contestações e manutenção da conexão, somente para administrador.
Informações auxiliares	Localização, privacidade e ajuda. A tela de estação permanece no código, mas não implica estações reais no conjunto atual.

Na revisão 3.0.4, Meu perfil mostra foto ou iniciais, nome, usuário e papel administrativo antes de qualquer menu. Minha conta reúne ações pessoais; Informações reúne chuva, localização, privacidade e ajuda; Administração fica abaixo. Não há modo de apresentação no login.

7 O caminho completo de um relato

Do formulário ao armazenamento local

`ReportScreen.tsx` coleta os campos e exige posição pelo GPS ou mapa. As categorias são `passable`, `partial` e `blocked`, exibidas como Leve, Moderado e Grave. Esses nomes descrevem a observação da pessoa e não uma classificação automática de risco calculada pela chuva.

Ao enviar, a tela chama `createReport` do store. Em `appState.ts`, o código valida a entrada, gera um `clientId` antes de falar com a API e monta o relato com estado `pending`. O identificador atual combina prefixo, horário e parte aleatória; não deve ser apresentado como um UUID criptográfico do relato mobile.

O store atualiza o estado e espera `localStorage.flush()`. A fila em `storage.ts` mantém as escritas em ordem. Só depois de a gravação terminar o app pode iniciar o envio ou confirmar sucesso local. Se o armazenamento falhar, a operação rejeita e a tela informa o problema.

Estados de envio: `pending` significa aguardando; `syncing`, tentativa em andamento; `synced`, confirmação válida do servidor; `failed`, tentativa sem confirmação ou com erro. Esses estados são diferentes da gravidade e do encerramento da ocorrência.

`observedAt` é o horário informado para a observação. No banco, `received_at` registra quando o servidor recebeu a ocorrência; após sincronizar, esse recebimento aparece como `createdAt` no objeto devolvido ao mobile. Um relato observado antes pode ser recebido depois por ter ficado offline. Datas com `Z` no JSON estão em UTC; a interface pode formatá-las no horário local.

Da fila à API e ao SQLite

Quando pode sincronizar, o store confere o servidor e a sessão. Para cada pendência própria, marca a tentativa e persiste. Se houver foto, envia a imagem primeiro e guarda o caminho retornado para poder retomá-la sem recriar o envio inteiro.

`services/api/hydroPulse.ts` prepara `POST /v1/reports`, e `client.ts` acrescenta URL, JSON, token, timeout e tratamento de erros. O cabeçalho `Idempotency-Key` tem o mesmo valor do `clientId` do corpo.

No servidor, `mobile-api.ts` valida os campos e a igualdade do cabeçalho com o identificador. Confere se o autor corresponde à sessão e se a imagem pertence à conta. Adapta a categoria do mobile para os nomes do banco e chama a rotina central de gravação.

`db/reports.ts` usa transação, verifica identificador e conteúdo e grava uma ocorrência nova ou recupera a anterior. Os detalhes do mobile, como endereço e imagem, são persistidos em `report_details` na transação do adaptador. Um relato novo pode gerar avisos internos para outras contas.

Da resposta à tela

O primeiro envio aceito retorna HTTP 201; repetir a mesma operação retorna HTTP 200. O cliente valida a resposta antes de marcar `synced` e persiste o resultado. Se a resposta não chegar, conserva o identificador e a pendência para repetir.

Falhas de rede são repetíveis. Para determinados erros permanentes de validação ou conflito, o app preserva a informação de falha e evita repetir indefinidamente o mesmo conteúdo inválido. Não troca silenciosamente a identidade para contornar um conflito.

Por que o reenvio não cria outra ocorrência

Imagine que o relato `report-exemplo-001` foi gravado. A rede caiu antes da resposta. O celular não sabe se a gravação aconteceu, então reenvia `report-exemplo-001` com os mesmos dados. O banco encontra o registro e o servidor devolve o original.

Se a mesma chave chegar com conteúdo diferente, o servidor responde HTTP 409. Isso evita sobrescrever uma observação que já pode ter sido recebida por outras pessoas. A unicidade protege a identidade; a comparação de conteúdo protege seu significado.

Explicação importante: há repetição de requisições, mas um único efeito lógico para aquela operação aceita. Não é uma garantia de “entrega exatamente uma vez” em toda a rede. É reenvio com efeito idempotente no servidor.

8 Sincronização e recuperação por cursor

`ConnectivityManager.tsx` observa a rede, a retomada do app e um intervalo de 60 segundos enquanto ele está ativo. Também existe Sincronizar agora. O store mantém uma promessa `activeSync`: se uma sincronização já está em andamento, as outras chamadas aproveitam aquela execução.

O fluxo principal é: aguardar gravações locais; consultar `/health`; conferir identidade do servidor e sessão; enviar relatos pendentes; buscar páginas do servidor; buscar dados oficiais; sincronizar operações e estado da comunidade. Falhas são mostradas, sem afirmar que a informação já chegou ao outro lado.

O que o cursor representa

O banco atribui uma sequência crescente `seq` aos relatos recebidos. O cursor marca até onde o cliente já processou a consulta incremental. Ele é a sequência do servidor, não o horário do celular.

Exemplo para explicar na banca: o cliente A parou no cursor 10. Enquanto ele estava sem rede, o cliente B criou o relato 11. Depois A enviou o próprio relato, que recebeu 12. Se A avançasse direto para 12 por causa do POST, perderia o relato 11. Por isso o POST não avança o cursor. A consulta de recuperação busca a página após 10 e processa os resultados em ordem.

`GET /v1/sync/pull?cursor=10` retorna relatos, novo cursor, `hasMore`, estações e `serverId`. Cada página tem limite de 100 relatos no contrato mobile. O cliente confere os dados e a ordem, mescla por identidade e persiste página e cursor na mesma gravação JSON. Se a resposta ou a página for inválida, não trata aquilo como recuperação concluída.

Identidade do banco e dados que mudam

`serverId` identifica a base, não a URL. Trocar a URL do túnel sem trocar o banco mantém a identidade. Uma base diferente exige rever cache, cursor e autenticação; não é seguro reaproveitar silenciosamente um cursor de outra base.

Confirmações, contestações, notificações, encerramento e visibilidade não dependem apenas de novos relatos. São atualizados também por `/v1/community`. Atualizações do autor e decisões de moderação têm seus próprios identificadores persistidos e podem ser reenviadas após perda de resposta.

Limite da execução: reconectar só resolve o envio se servidor, sessão e validação também estiverem disponíveis. A fila conserva os dados; ela não torna uma operação inválida automaticamente aceitável nem garante trabalho irrestrito em segundo plano.

9 API e contratos HTTP

Uma API é o conjunto de operações que o servidor oferece aos clientes. Uma rota combina método e caminho; por exemplo, GET consulta e POST envia uma operação. JSON é o formato de dados usado na maior parte das mensagens. HTTP sobre TLS corresponde ao acesso HTTPS usado pelo celular.

O servidor roda em Node.js. `index.ts` abre a base, prepara a conta inicial quando não existem contas, importa a cópia oficial se a tabela estiver vazia, inicia manutenção e sobe o Express. `app.ts` monta os middlewares e os roteadores. `config.ts` trata caminho do banco, porta e host.

Rotas que você deve reconhecer

Método e caminho	Função e exigência principal
GET /health	Estado, versão, capacidades e identidade da base. Público.
POST /v1/auth/register e /v1/auth/login	Criar conta e entrar; retornam sessão. Cadastro também retorna chave de recuperação.
GET e PATCH /v1/auth/me	Consultar ou atualizar o próprio perfil. Exigem sessão.
POST /v1/auth/logout	Revogar a sessão apresentada.
POST /v1/auth/recover	Recuperar acesso com usuário, chave e nova senha.
POST /v1/auth/password e /v1/auth/recovery-code	Trocar senha ou gerar outra chave, confirmando senha atual.
POST /v1/auth/delete	Excluir a própria conta com senha e confirmação EXCLUIR.
GET /v1/observations e /v1/stations	Histórico oficial e estado da fonte; estações atualmente vazias. Públicos.
POST /v1/media e GET /v1/media/:id	Enviar imagem autenticado e recuperar conteúdo disponível pelo identificador.
POST /v1/reports	Criar ou reconhecer reenvio de relato. Exige sessão e chave idempotente.
GET /v1/reports e /v1/sync/pull	Consultar relatos e recuperar páginas conforme a visibilidade permitida.
POST /v1/reports/:id/confirm	Confirmar relato de outra pessoa uma vez por conta.
POST /v1/report-disputes	Enviar contestação com identificador e justificativa.
POST /v1/reports/:id/state	Autor atualiza situação e nota com identificador de operação.
GET /v1/community e POST /v1/notifications/read	Recuperar estado comunitário e marcar aviso da própria conta como lido.
GET /v1/moderation e POST /v1/moderation/:id	Listar e decidir contestações. Exigem papel de moderador.

`PATCH /v1/reports/:id` retorna 405 para relatos enviados: o conteúdo original é imutável. Encerrar uma ocorrência usa a rota de estado, que registra um evento separado. As rotas canônicas `/reports` e `/observations` também existem e apoiam o núcleo e os testes; não representam outro banco ou uma segunda aplicação.

Exemplo de criação

O exemplo abaixo é didático; `ID_DA_CONTA` representa o identificador da sessão de ensaio. A chamada real envia também `Authorization: Bearer TOKEN`, além de `Content-Type: application/json`. Não projete um token real.

```
POST /v1/reports
Idempotency-Key: report-exemplo-001

{
  "clientId": "report-exemplo-001",
  "authorId": "ID_DA_CONTA",
  "category": "partial",
  "description": "DEMONSTRAÇÃO - passagem parcial",
  "address": "Ponto de ensaio em Santos",
  "coordinates": {
    "latitude": -23.9608,
    "longitude": -46.3336
  },
  "observedAt": "2026-10-26T12:00:00.000Z",
  "locationMode": "map"
}
```

As coordenadas e a data do exemplo não são uma ocorrência real nem uma consulta automática do endereço. Na demonstração, escolha e identifique seu próprio ponto de ensaio.

Respostas e validação

HTTP 200 significa operação consultada ou repetição reconhecida; 201, criação; 400, entrada inválida; 401, ausência ou rejeição de autenticação; 403, ação sem permissão; 404, recurso indisponível; 405, método não permitido; 409, conflito; 413, corpo grande demais; 429, limite de requisições. A mensagem de erro normalmente usa `error.code` e `error.message`.

O cliente aplica timeout padrão de 10 segundos e 30 segundos para upload. O servidor valida novamente: TypeScript não impede que alguém envie JSON manualmente. SQL parametrizado separa os valores dos comandos; limites de tamanho e tentativas reduzem abuso. CORS controla origens web permitidas, mas não autentica o usuário nem substitui o token.

10 Banco de dados central

SQLite é um banco relacional armazenado em arquivo. As tabelas são criadas com `CREATE TABLE IF NOT EXISTS`; migrações acrescentam campos necessários preservando os dados. Reiniciar a API não executa a limpeza de apresentação.

O modo WAL permite que leituras e gravação coexistam em determinadas condições; ainda há um escritor por vez. `busy_timeout` está em 5 segundos e as chaves estrangeiras estão habilitadas. As transações de escrita importantes usam execução imediata para coordenar a alteração no arquivo.

Tabela ou conjunto	O que guarda
app_metadata	Identidade `serverId` e metadados da aplicação.
observations e source_state	Medições, origem, unidade, abrangência, datas e resultado da última tentativa de consulta.
reports e report_details	Núcleo da ocorrência e sequência; endereço, autoria e caminho da imagem do contrato mobile.
accounts e sessions	Identidade, nome de acesso, hash da senha, papel e hash do token com expiração.
media e hidden_media	Bytes das imagens, proprietário, MIME e bloqueio de entrega por moderação.

Tabela ou conjunto	O que guarda
confirmations	Relação relato-conta com chave composta para não confirmar duas vezes.
disputes	Justificativas, evidências e resultado da análise de contestações.
report_states e report_state_events	Situação atual, visibilidade e histórico das atualizações do autor.
moderation_actions	Identificador, responsável e justificativa de cada decisão.
notifications	Avisos por conta, com chave de evento única e data de leitura.

Restrições que sustentam o comportamento

`reports.id` é único; a sequência `seq` ordena recebimentos. A medição tem unicidade por fonte e chave de registro, para importar novamente sem duplicar a mesma data e abrangência. Confirmações têm chave composta por relato e conta. Notificações têm unicidade de conta e evento. Esses limites no banco complementam as verificações da API.

Consultas de leitura para explicar

Use um visualizador SQLite na base de ensaio apropriada. As consultas abaixo são apenas de leitura e não precisam expor dados de sessão, hashes ou imagens:

```
SELECT COUNT(*) AS total FROM reports;

SELECT seq, id, category, observed_at
FROM reports ORDER BY seq DESC LIMIT 5;

SELECT source, observation_date, value, unit
FROM observations ORDER BY observation_date DESC LIMIT 5;

SELECT report_id, lifecycle, visibility
FROM report_states;
```

O arquivo `documentos/ESTRUTURA_DO_BANCO.sql` documenta a estrutura sem distribuir o banco de ensaio com contas, sessões e fotos. O inicializador da API cria a base; não é necessário importar um dump particular.

11 Conta segurança e privacidade no código

Autenticação e autorização

Autenticação verifica quem está usando a API; autorização verifica o que essa conta pode fazer. O login normaliza o nome de acesso e confere a senha. A API usa um token aleatório opaco, não JWT. O token é enviado como Bearer nas requisições; o servidor conserva seu hash e uma expiração de 30 dias.

`security.ts` gera hash de senha com scrypt e salt aleatório. A comparação usa `timingSafeEqual`. Hash não é uma senha criptografada que o servidor precisa descriptografar: a conferência recalcula uma representação e compara. A aplicação real não persiste a senha informada no cadastro/login; o campo de compatibilidade devolvido ao cliente é vazio.

No Android, `deviceStorage.native.ts` separa o token do JSON e o grava em SecureStore. Ao restaurar, `sessionStorage.ts` confere a correspondência com conta e identidade da base. Os demais dados continuam no AsyncStorage, sem promessa de criptografia integral. A implementação web usa armazenamento próprio da plataforma e não deve ser apresentada como SecureStore Android.

Recuperação e alteração da conta

Cadastro, recuperação e troca de senha podem apresentar uma chave secreta. Só o resumo da chave fica no servidor; o texto aparece ao usuário para que ele o guarde. A chave não entra no estado persistido do app. Gerar outra invalida a anterior. Recuperar ou trocar senha revoga as sessões anteriores e devolve uma sessão nova. Não há envio de e-mail ou SMS.

O nome de acesso fica em `login_name` e na chave normalizada; o nome exibido pode mudar sem mudar o login. Telefone, endereço e foto são opcionais no uso conectado. Os contatos do perfil não são listados nos relatos públicos.

A conta administrativa inicial

`presentation.ts` cria Roger com papel `moderator` se a base não tiver contas. O preparo do ensaio definiu a senha inicial 12345678, gravada no banco como hash. É uma configuração conhecida do banco acadêmico, não um modelo de credencial para abertura pública.

O aplicativo mantém cadastro e contas comuns. A limpeza já realizada deixou uma base inicial sem relatos, preservando medições oficiais; depois do uso, novos dados podem existir. `resetPresentationData` é uma operação administrativa explícita. Reiniciar a API não apaga relatos; não execute o comando de reset durante a apresentação.

Exclusão e acesso aos dados

Excluir conta exige sessão, senha atual e confirmação EXCLUIR. O servidor remove os dados próprios e mantém marcadores de remoção dos identificadores dos relatos, impedindo que um cliente antigo recrie o mesmo ID. Outros aparelhos precisam sincronizar para descartar cópias já recebidas. Backups têm retenção e exigem cuidado ao restaurar exclusões posteriores.

Os relatos visíveis são compartilhados e imagens disponíveis podem ser recuperadas pelo caminho de mídia. Portanto, token de escrita não significa que todas as fotos sejam privadas. Conteúdo ocultado deixa de ser entregue publicamente pelo servidor, mas isso não apaga automaticamente cópias já vistas ou em cache em outro dispositivo.

12 Mapa localização e imagens

O mapa de ruas

`map/document.ts` monta um documento HTML com Leaflet empacotado no projeto. As ruas são tiles de `tile.openstreetmap.org`, com atribuição de autoria. `CityMap.native.tsx` abre o documento em WebView; `CityMap.web.tsx` usa iframe. Mensagens estruturadas ligam o documento às telas.

`map/geo.ts` valida coordenadas e monta os pontos. No mapa de relatos, verde indica Leve, laranja Moderado, vermelho Grave e cinza uma ocorrência encerrada. A busca e os filtros controlam os itens exibidos; encerradas ficam ocultas até serem habilitadas. A câmera inicial em Santos é apenas o enquadramento, não um relato.

Textos de relatos entram nos elementos do popup com `textContent`. A serialização protege caracteres ao transportar os dados, e o documento restringe origens com uma política de conteúdo. Esses cuidados reduzem a possibilidade de tratar uma descrição como código executável.

GPS e seleção de ponto

`services/location.ts` pede permissão no momento da ação, verifica o serviço de localização e limita a espera. Se o usuário negar, o app informa o motivo e permite escolher um ponto no mapa. Não há rastreamento contínuo em segundo plano.

Um marcador só representa a posição do relato quando `locationMode` é GPS ou mapa e as coordenadas são válidas. O endereço é uma descrição para leitura humana; alterar esse texto não desloca automaticamente o ponto. Geocodificação reversa, quando disponível, auxilia a descrição, sem substituir as coordenadas escolhidas.

O código admite compatibilidade com posição por referência em registros antigos e no núcleo de testes. A tela atual exige GPS ou seleção no mapa para novos relatos. Não confunda um dado legado com a regra do formulário atual.

Fotos e persistência

`selectPhoto.ts` usa Expo Image Picker, prepara a imagem e a transforma em uma URI de dados base64. Assim, o relato pendente não depende apenas de um caminho temporário de outro aplicativo. O seletor pode ser cancelado e a foto pode ser removida antes de enviar.

O upload aceita JPEG, PNG ou WebP de até 2 MB, com validação do formato e dos bytes pelo servidor. No Android, a exportação usada no seletor prepara JPEG. A API armazena os bytes em `media` e devolve `/v1/media/IDENTIFICADOR`. A identidade da mídia depende do proprietário e do conteúdo, permitindo reconhecer o mesmo upload.

`AppImage.tsx` combina esse caminho relativo com a URL atual da API. Por isso mudar a URL do túnel para a mesma base não exige reescrever todos os caminhos de fotos. Salvar o ponto e o relato offline não significa ter um pacote completo de ruas offline: o mapa depende de internet ou cache disponível.

13 Dados oficiais de chuva

A fonte efetivamente usada é a tabela Dados de Pluviometria da Prefeitura de Santos. A cópia incluída em `data/official/santos-2026-10.json` contém 22 datas, de 01 a 22/10/2026, total de 169,7 mm. `mobile/src/data/official-snapshot.json` oferece um histórico inicial mesmo antes de uma nova sincronização.

`import/santos.ts` busca a página HTTPS, lê datas e índices, trata vírgula decimal e confere estrutura, datas válidas e, quando publicado, o acumulado. Uma célula vazia ou traço vira ausência de valor, não zero inventado. Se a estrutura não for reconhecida, a importação falha e mantém o conjunto anterior.

`import/official.ts` valida os registros, normaliza metadados e faz inserção ou atualização por fonte e chave de registro. A chave considera abrangência, variável, período e data. Importar de novo não cria uma segunda medição do mesmo registro. O servidor não inventa horário para um dado que só tem precisão de dia.

Três datas que você deve diferenciar

Informação	Significado
Data da medição	Data publicada para o índice de chuva, como 22/10/2026.
Última consulta à fonte	Quando o servidor conseguiu consultar/importar a publicação. Pode ser posterior à medição.
Última sincronização do app	Quando o celular completou sua atualização com a API. Não torna a medição mais recente.

> **Resposta preparada sobre 22/10:** “A cópia usada no projeto terminava nessa data, e a consulta registrada em 25/10 ainda não trouxe medições posteriores. O sistema atualiza a consulta, mas só pode mostrar datas que a fonte publicou. Por isso separamos a data do dado e a data da consulta.”

O serviço tenta atualizar ao iniciar a manutenção e a cada 6 horas. `source_state` guarda tentativa, sucesso e erro. Uma falha não substitui a chuva por valores fictícios. A fonte é municipal e não fornece coordenadas de estações no conjunto adotado; `/v1/stations` retorna vazio. Não afirme que foram integrados sensores, estações em tempo real ou o Cemaden nesta entrega.

Unidade mm e organização por data seguem a interpretação documentada da publicação. Não apresente o índice como um sensor de 24 horas com horário exato, previsão ou confirmação de alagamento.

14 Confirmações contestações e encerramento

Uma conta pode confirmar o relato de outra pessoa. O backend impede confirmar o próprio relato e limita a confirmação a uma por conta e ocorrência. O cliente guarda a pendência antes do envio, e a chave composta no banco impede repetir o efeito.

A contestação registra justificativa de 10 a 500 caracteres e foto opcional. Tem identificador próprio persistido para reenvio. Ela informa discordância ou contexto; não apaga automaticamente o relato. A moderação analisa o conteúdo com uma conta autorizada.

O moderador pode manter ou ocultar o relato com justificativa. A operação exige papel `moderator` no servidor, não apenas um botão escondido na tela. Decisões têm `operationId`, registro no banco e aviso às partes. A interface permite revisar uma decisão com uma nova operação.

Três dimensões independentes

Dimensão	Valores principais	Pergunta que responde
Envio no celular	pending, syncing, synced, failed	O servidor confirmou o recebimento?
Situação da ocorrência	active ou resolved	O autor considera a situação em andamento ou encerrada?
Visibilidade da publicação	visible, hidden ou removed	O conteúdo pode continuar sendo mostrado?

Encerrar muda `lifecycle` e guarda nota e evento do autor. Ocultar muda `visibility` por decisão administrativa. Restaurar a publicação conserva o encerramento e sua nota; moderar não reabre uma ocorrência automaticamente. Um relato já enviado mantém o conteúdo original; novas informações ficam em operações separadas.

Os avisos sobre relatos e interações ficam em `notifications`, com uma chave de evento por destinatário para evitar duplicação do mesmo aviso. Chegam ao app na sincronização. Não há push do Android nem alerta emitido pela Defesa Civil.

15 Como apresentar o código sem se perder

Abra os arquivos na ordem abaixo. Leia a responsabilidade e siga a chamada principal. Mostre um trecho por vez e evite ler todo o arquivo na tela.

Ordem	Arquivo	O que apontar
1	mobile/App.tsx	Hidratação, sessão, abas e restrição das rotas administrativas.
2	mobile/src/screens/ReportScreen.tsx	Estado do formulário e chamada ao store ao enviar.
3	mobile/src/store/useAppStore.ts	Criação do store conectado e escolha do armazenamento por plataforma.
4	mobile/src/store/appState.ts	createReport, gravação antes do envio, activeSync e recuperação por cursor.
5	mobile/src/store/storage.ts	Ordem das gravações e flush.
6	mobile/src/services/api/hydroPulse.ts	Corpo do POST, upload e Idempotency-Key.
7	mobile/src/services/api/client.ts	URL, token, timeout, JSON e erros.
8	server/src/app.ts	Montagem do servidor, middlewares e roteadores.
9	server/src/mobile-api.ts	Contrato mobile, permissão do autor e adaptação das categorias.
10	server/src/db/reports.ts	Transação, identificador único e comparação do conteúdo.
11	server/src/db/database.ts	Tabelas, WAL, chaves estrangeiras e identidade da base.
12	mobile/src/components/ConnectivityManager.tsx	Gatilhos de sincronização ao reconectar e retomar.

Como ler os trechos de TypeScript

Trecho ou recurso	Significado prático
import e export	Um arquivo usa uma função de outro ou a disponibiliza. É a ligação entre módulos.
type e interface	Descrevem os formatos esperados pelo TypeScript. Não substituem a validação de JSON que chegou pela rede.
useState	Guarda um valor da tela e permite alterá-lo, como a descrição digitada. Esse estado sozinho não é persistência.
useAppStore	Lê ou modifica o estado compartilhado. As ações do store coordenam persistência e API.
async e await	Organizam tarefas assíncronas: aguardar a gravação local ou a resposta de rede antes de continuar aquela função.
try catch finally	Tratam sucesso, falha e limpeza, como retirar o indicador de carregamento mesmo após um erro.
map e filter	Transformam uma lista ou selecionam seus itens, por exemplo separar relatos pendentes.
router.post e res.json	Registram uma operação da API e devolvem sua resposta em JSON.

Por exemplo, `const user = useAppStore((state) => state.user)` seleciona a conta do estado compartilhado para a tela. `await localStorage.flush()` espera a gravação já enfileirada. `safeParse(req.body)` confere a entrada recebida; `db.prepare(...).run(...)` executa uma escrita SQL com os valores enviados separadamente. Não são quatro formas de guardar a mesma coisa: leitura de estado, persistência local, validação e escrita no servidor cumprem papéis diferentes.

Outros módulos que completam o projeto

No celular: `store/serverAccounts.ts` implementa ações conectadas de conta; `store/community.ts` trata interações, atualizações e moderação; `store/persistedState.ts` valida o estado recuperado; `store/deviceStorage.native.ts` e `sessionStorage.ts` separam a sessão protegida. `useVisibleReports.ts` filtra conteúdo e mantém pendências restritas à conta autora na interface.

Interface e contratos: `screens/` contém as telas da seção 6; `components/` reúne cabeçalho, botões, cartões, imagens e mapa; `theme/` centraliza cores, medidas e tipografia; `types/domain.ts` e `types/navigation.ts` descrevem dados e rotas; `utils/validation.ts` confere dados em execução, e outros utilitários cuidam de formato, busca e diálogos.

Mapa e recursos: `map/geo.ts`, `map/document.ts` e `map/useMapData.ts` separam dados geográficos, documento do Leaflet e comunicação com as telas. `services/location.ts` trata GPS; `services/selectPhoto.ts`, seleção de imagem. `mobile/scripts/prepare-map.mjs` prepara os recursos do Leaflet para o pacote.

No servidor: `community.ts` reúne contas, sessões, mídia, confirmações, contestações e avisos; `security.ts` trata senhas e chaves; `product.ts` reúne recuperação, exclusão, estados e moderação; `operations.ts` faz atualização e backup; `import/` trata os dados reais; `validation/report.ts` valida o contrato canônico; `cli.ts` fornece os comandos administrativos e de demonstração; `presentation.ts` prepara a conta inicial e a limpeza explícita.

Operação e entrega: Em Windows, scripts/ambiente-windows.ps1 prepara as ferramentas e é chamado pelos dois CMDs. `scripts/setup.mjs` prepara dependências; `scripts/presentation.mjs` coordena API e túnel no notebook; `scripts/prepare-mobile-data.mjs` atualiza a cópia inicial do celular; `scripts/check-release.mjs` verifica condições de distribuição pública. Os `.cmd` da raiz são atalhos Windows. `Dockerfile`, `render.yaml` e `mobile/eas.json` são configurações de implantação/build; sua existência não prova que a hospedagem ou a loja foram ativadas.

Compatibilidade e ensaio: As suítes de testes não acompanham este pacote enxuto. `data/mock.ts` e ramos de simulação permanecem para desenvolvimento e testes; o APK entregue usa `EXPO_PUBLIC_USE_MOCKS=false`. A pasta histórica mobile-old foi retirada desta cópia; o aplicativo atual está em mobile. Arquivos de configuração, assets e lockfiles completam a reprodução do build, mas não substituem os fluxos acima.

16 Validação registrada

A revisão 3.0.4 registrou 93 testes automatizados aprovados: 22 servidor, 34 mobile e 37 integração/configuração. Esta cópia enxuta não contém as suítes nem o comando npm test. Os resultados se referem à rodada registrada antes da organização deste pacote; as suítes completas continuam no ZIP anterior.

TypeScript, exportação web, compilação Android, assinatura v2 e alinhamento de 16 KB foram conferidos na entrega original. A revisão visual 3.0.4 teve 30 cenários locais com componentes reais via React Native Web, incluindo configuração por código, nome longo e fonte ampliada por CSS. A área do mapa foi neutra nesse ensaio visual, sem nova verificação de ruas/GPS ou inspeção física do Android.

Os ensaios anteriores usaram um Samsung SM-S711B com Android 16, operação manual e contraparte HTTP. Não há declaração de dois celulares Android nem de compatibilidade universal. Os detalhes essenciais estão em documentos/VERIFICACOES.md.

Comandos mantidos para apresentar

Na pasta codigo, você pode usar:

```
npm run typecheck
npm run demo:idempotency
```

O primeiro confere tipos do código; o segundo demonstra a perda de resposta depois da gravação, em banco temporário isolado, com duas tentativas e um registro. Essa demonstração faz parte da apresentação e foi mantida. Não equivale à execução das suítes removidas. O banco usado pelo celular não é apagado por esse comando.

17 Como preparar a execução amanhã

Antes de sair para a aula

1. Separe o APK 3.0.4, o ZIP completo, os slides e este roteiro. O APK é Android arm64, com versão mínima Android 7; não é um pacote universal para qualquer aparelho ou iOS.

2. Execute PREPARAR_DEPENDENCIAS.cmd com internet. Ele verifica Node.js 24 e cloudflared, baixa as ferramentas ausentes de fontes oficiais e instala as dependências. As ferramentas locais ficam em codigo/.runtime/ferramentas. Depois execute INICIAR_APRESENTACAO.cmd e aguarde CONEXÃO VERIFICADA.

3. Mantenha a janela, o notebook e a conexão ativos. O telefone acessa a URL pública HTTPS; ele não acessa o `localhost` do notebook diretamente. `localhost` no próprio celular apontaria para o celular.

4. Abra o app, entre na conta de ensaio, sincronize e confira a fonte, o mapa e um ponto. Não deixe o primeiro teste dessa combinação para o momento da apresentação.

5. Prepare uma foto pequena de ensaio e mantenha os arquivos principais abertos no editor. Ensaiem a divisão das falas e a mudança do modo avião.

Endereço do serviço

O APK tem um endereço inicial e permite indicar outra URL antes de entrar. No rodapé do login, toque em Configurar conexão, informe 180504 e escolha Liberar configuração. Cole a URL HTTPS base, toque em Testar e salvar conexão e depois Voltar para entrar. A liberação termina ao sair da tela. O administrador também tem o ajuste no Perfil. A URL de trycloudflare.com é temporária e pode mudar ao reiniciar o túnel.

Se o endereço mudar, uma instalação nova pode receber a URL pelo login, usando o código 180504. O app confere a identidade e as capacidades do serviço antes de salvar. Esse código libera apenas a configuração local: não entra em uma conta nem concede moderação. Uma conta comum já autenticada precisa sair para usar esse acesso; Roger também pode ajustar pelo Perfil.

O iniciador escolhe `DB_PATH` se definido; no projeto original pode conservar a base de ensaio existente. Em uma cópia nova no Windows, o padrão do iniciador é `%LOCALAPPDATA%/HydroPulse/server.db`. Isso é diferente do padrão ao iniciar o servidor diretamente por seu próprio script. Use o iniciador para repetir o preparo combinado e confira o caminho mostrado.

Se uma dependência cair durante a fala

Se a API não responder, mostre o dado já salvo e a pendência, explique a falha parcial e retome o iniciador quando possível. Não apresente a pendência como publicada. Se as ruas não carregarem, explique a dependência de tiles e use a lista de relatos e as coordenadas já salvas. A seleção de ponto para o ensaio deve ser preparada enquanto o mapa está acessível.

O comando de idempotência usa banco isolado e pode demonstrar o núcleo no computador sem alterar a base da aula. Não execute reset, exclusão da conta principal ou comandos de limpeza para “tentar fazer funcionar” durante a apresentação. Cadastro, senha e recuperação podem ser explicados sem projetar credenciais reais.

18 Operação manutenção e limites da entrega

O que o servidor faz em manutenção

`operations.ts` programa consulta da fonte a cada 6 horas e backup a cada 24 horas, além de uma tentativa inicial após iniciar. Uma operação do mesmo tipo não roda simultaneamente consigo mesma. O encerramento do processo aguarda trabalhos em andamento antes de fechar a base.

O backup usa a API de backup do SQLite para obter uma cópia consistente, inclusive com WAL. A retenção padrão mantém três arquivos do aplicativo. Backup no mesmo disco não protege contra perda daquele volume; uma implantação real precisa de cópia externa e teste de restauração.

O que está pronto e o que exige implantação

Há aplicativo, API, dados oficiais, testes e configuração de implantação. O uso escolhido para a aula depende do notebook e do túnel. A hospedagem permanente não foi ativada. O pacote Android atual tem assinatura de desenvolvimento; distribuição pública exige chave do titular e preparo próprio.

`check-release.mjs` rejeita condições como URL temporária, mocks ou falta de credenciais privadas de assinatura para distribuição pública. Esse controle não é uma rota da API nem um botão do usuário. A execução de aula tem alcance diferente da publicação em loja com disponibilidade contínua.

Antes de abrir a um público amplo, ainda é necessário definir a hospedagem com volume persistente, testar redeploy e restauração, preparar a assinatura de distribuição, organizar responsáveis pela moderação e publicar as informações de contato e privacidade do operador. Não foram contratados serviços nem inventados esses dados.

O sistema não faz previsão de alagamentos, não classifica automaticamente a veracidade de relatos, não integra sensores físicos, não tem push e não oferece mapa integralmente offline. A proposta original delimita esses recursos e define o resultado como aplicação acadêmica funcional, não serviço municipal de emergência.

19 Revisão rápida e glossário

Um caminho para memorizar

Formulário → store → gravação local → fila → API → validação → transação SQLite → resposta → estado sincronizado. Para recuperar o que outros enviaram: **cursor anterior → consulta paginada → validação → mescla e persistência → próximo cursor.**

Termo	Explicação no projeto
API e endpoint	Operações do servidor acessadas por métodos e caminhos HTTP.
Payload e JSON	Dados de uma mensagem e seu formato textual.
Cache	Cópia de informação já recebida para consulta local.
Persistência e hidratação	Guardar além da memória e recuperar ao abrir o app.
Fila de envio	Operações salvas que ainda aguardam confirmação remota.
Idempotência	Repetir a mesma operação aceita sem criar outro efeito lógico.
Cursor	Marcador da sequência de relatos que o cliente já processou.
Token	Credencial temporária da sessão enviada nas requisições autenticadas.
Hash e salt	Representação para conferência da senha e valor aleatório usado em sua derivação.
Transação e COMMIT	Conjunto consistente de alterações e confirmação da gravação.
Falha parcial	Uma parte fica indisponível enquanto outras continuam funcionando.
Consistência eventual	Estados podem divergir durante desconexão e se atualizar quando há comunicação válida.
Tile	Uma das imagens que compõem o mapa de ruas.
Mock	Implementação simulada usada em desenvolvimento e testes, desativada no APK entregue.

Frases tecnicamente corretas

“O telefone usa AsyncStorage; o SQLite é central, no servidor.” “Os dados de chuva são oficiais e históricos, com datas preservadas.” “A ocorrência tem um identificador estável antes do primeiro envio.” “Somente o fluxo de recuperação avança o cursor.” “A senha usa hash scrypt, e o token Android fica no SecureStore.” “O mapa conserva pontos salvos, mas as ruas precisam de internet ou cache.”

Base documental e técnica deste roteiro

A proposta original é `documentos/PROPOSTA_APS_ORIGINAL.pdf`, cópia do PDF fornecido pelo grupo. As responsabilidades e o escopo acima foram confrontados com `mobile/App.tsx`, telas, store, serviços, mapa e tipos; `server/src/` com rotas, banco, segurança, importação e operação; scripts de preparo; configurações; e suítes de testes.

Para conferir evidências, consulte documentos/VERIFICACOES.md. O roteiro e o relatório mantêm a explicação dos fluxos; os históricos e guias repetidos ficaram no pacote completo. Documentos antigos podem registrar versões e quantidades anteriores; os números desta revisão são 3.0.4 no app, 3.0.1 na API e 93 testes. A página institucional indicada nos registros oficiais é https://www.santos.sp.gov.br/?q=pluviometria-tabela.

Se o Node.js foi preparado somente na pasta local, comandos manuais podem ser executados dentro de codigo com powershell -NoProfile -ExecutionPolicy Bypass -File scripts/ambiente-windows.ps1 -Acao Demonstrar. Para verificar tipos, use -Acao Tipos.

O roteiro separa o que estava planejado, o que o código implementa, o que foi observado nos ensaios e o que ainda depende de implantação. Use essa mesma distinção ao responder ao professor.

20 Dez perguntas que o professor pode fazer

1 Por que o Hydro Pulse é um sistema distribuído se existe apenas um servidor

Resposta: Porque aplicativo, servidor e fontes externas são componentes independentes que trocam informações pela rede. O celular mantém estado local, pode ficar desconectado e depois precisa recuperar o estado compartilhado. O servidor pode continuar recebendo relatos de outros clientes enquanto um aparelho está offline. O projeto demonstra falhas parciais, reenvio, persistência e recuperação; não precisa de microsserviços ou vários servidores para ter essas características.

Se pedir para mostrar: abra `ConnectivityManager.tsx` e `appState.ts` e explique a pendência local enquanto a API não pode ser acessada.

2 O que acontece se o servidor gravar o relato mas a resposta não chegar ao celular

Resposta: O celular conserva o identificador criado antes do primeiro envio e repete a operação. Na API mobile, `Idempotency-Key` deve ser igual ao `clientId`. O banco encontra a identidade já gravada e o servidor devolve o mesmo relato se o conteúdo for igual, sem outra ocorrência. Se os dados forem diferentes com a mesma chave, há conflito 409. Assim há várias tentativas possíveis, com um único efeito lógico para a mesma operação aceita.

Se pedir para mostrar: execute `npm run demo:idempotency`: duas tentativas e uma linha. A regra central está em `server/src/db/reports.ts`.

3 Como vocês garantem que o relato offline não some quando o aplicativo é fechado

Resposta: O store grava o relato no AsyncStorage e espera o término da fila de escrita por `flush` antes do envio ou da confirmação de sucesso local. A imagem preparada também acompanha a pendência. Ao abrir novamente, o app recupera e valida o estado; uma tentativa interrompida pode ser retomada com a mesma identidade. Isso cobre o fechamento do aplicativo nos cenários ensaiados, mas não protege contra desinstalação, limpeza dos dados ou defeito do armazenamento.

Se pedir para mostrar: salve sem rede, feche e reabra. No código, mostre `createReport`, `storage.ts` e `persistedState.ts`.

4 Por que usar cursor em vez de consultar apenas a data da última atualização

Resposta: O cursor acompanha a sequência de recebimento no servidor e evita depender do relógio do telefone. Se A parou em 10, B criou 11 e A depois enviou 12, avançar pelo resultado do POST faria A perder o relato de B. O app busca páginas após 10 e só atualiza o cursor pelo fluxo de recuperação, com a página validada e persistida. Estados que mudam sem novo relato, como moderação e confirmações, são consultados também em `/v1/community`.

Se pedir para mostrar: a consulta `seq > cursor`, em `db/reports.ts`, e o laço de `pullSync` em `appState.ts`.

5 Os dados são reais e por que as medições da cópia terminam em 22 de outubro

Resposta: Sim, a cópia foi importada da tabela da Prefeitura de Santos e conserva a fonte e as datas publicadas. Ela contém 22 registros de 01 a 22/10/2026, total de 169,7 mm. A consulta registrada em 25/10 ainda não trouxe dias posteriores. Uma consulta recente não transforma uma medição antiga em dado atual. O servidor tenta consultar a fonte a cada 6 horas e preserva o histórico quando há falha ou ausência de publicação nova. Esses índices não comprovam alagamento de uma rua.

Se pedir para mostrar: a tela Medições reais de chuva, `data/official/santos-2026-10.json` e o importador em `server/src/import/`.

6 A proposta previa SQLite no celular por que a implementação usa AsyncStorage

Resposta: A proposta inicial permitia ajustes de tecnologia preservando seus objetivos. O mobile utilizado já trabalhava com AsyncStorage, e a implementação manteve esse armazenamento com JSON, validação e fila ordenada de gravações. Isso atende à persistência local e à retomada das pendências sem adicionar outra camada para este escopo. O SQLite está no servidor e oferece transações e restrições de unicidade. Não afirmamos que AsyncStorage execute SQL ou tenha os mesmos recursos relacionais.

Se pedir para mostrar: `deviceStorage.native.ts`, `storage.ts` e `server/src/db/database.ts`.

7 Como uma pessoa é impedida de enviar por outra conta ou acessar a moderação

Resposta: A API autentica o token de sessão e confere permissões no servidor. O autor enviado e o proprietário da imagem precisam corresponder à conta autenticada. A senha é conferida por hash scrypt com salt; o banco conserva hash do token e expiração. Moderação exige papel `moderator`, mesmo que alguém tente chamar a rota diretamente. No Android, o token fica no SecureStore. Esconder o botão é apenas parte da interface; a autorização decisiva acontece na API.

Se pedir para mostrar: `requireAccount` em `community.ts`, a conferência no `mobile-api.ts` e `requireModerator` em `product.ts`.

8 Um relato pode ser alterado depois de enviado e qual a diferença entre encerrar e ocultar

Resposta: O conteúdo enviado não é sobrescrito pelo PATCH: a API responde 405. O autor registra uma atualização separada para informar situação ativa ou encerrada e uma nota. A moderação pode manter ou ocultar a publicação por meio de uma decisão justificada. Encerramento pertence a `lifecycle`; visibilidade pertence a `visibility`. Restaurar um relato ocultado mantém o encerramento anterior e sua nota. As operações têm identidades próprias para reconhecer reenvios e manter histórico.

Se pedir para mostrar: `report_states`, `report_state_events`, `moderation_actions` e as rotas de estado e moderação em `product.ts`.

9 Como vocês provaram o funcionamento e não apenas construíram telas

Resposta: A revisão registra 93 testes automatizados entre servidor, mobile e integração, além de TypeScript e compilação. Há testes HTTP de resposta perdida após gravação, retomada do estado, contas, fotos, operações e recuperação. Um Samsung Android foi operado manualmente em ensaios anteriores, com contraparte HTTP e conferência de dados no servidor. A interface 3.0.4 teve 30 cenários visuais locais, mas não uma nova inspeção física. Portanto, distinguimos evidência automatizada, ensaio real e o que ainda não foi coberto.

Se pedir para mostrar: mostre documentos/VERIFICACOES.md e a demonstração de idempotência. As suítes estão no pacote completo.

10 O sistema já pode ser publicado para toda a população e o que ainda falta

Resposta: O aplicativo e a API estão implementados para os fluxos apresentados, mas a execução da aula usa notebook e túnel temporário. Para disponibilidade independente é necessário implantar a API com endereço fixo e armazenamento persistente, validar restauração e redeploy, preparar assinatura de distribuição e definir operação e moderação. Também faltam testes mais amplos de dispositivos e acessibilidade. As ruas dependem de internet ou cache e os avisos são internos, sem push. O resultado atende ao propósito acadêmico delimitado na proposta; não é um serviço oficial de emergência nem previsão de alagamento.

Se pedir para mostrar: `Dockerfile`, `render.yaml`, `scripts/check-release.mjs` e os guias de implantação e assinatura, deixando claro que configuração preparada não é hospedagem ativada.